Hibernate

Instructor - https://in.linkedin.com/in/suryakantsurve

Maven

What is Maven?

- Apache Maven is a powerful build automation and project management tool primarily used for Java-based projects. It simplifies the software development process by handling:
  - Dependency management
  - Build processes (compile, test, package, deploy)
  - Project documentation
  - Reporting

POM (Project Object Model)

- The heart of Maven - an XML file (pom.xml)
- Contains project configuration, dependencies, and build settings

```
<project>
  <modelVersion>4.0.0</modelVersion>
    <!-- Project coordinates (like an address) -->
  <groupId>com.mycompany</groupId>
  <artifactId>my-app</artifactId>
  <version>1.0.0</version>
    <!-- Dependencies -->
  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.13.2</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
```

Dependencies & Repositories

- Central Repository
  - Maven's default public repository (contains millions of libraries)

- Local Repository
  - ~/.m2/repository on your machine (caches downloaded dependencies)

- Private Repositories
  - Companies often set up their own (Nexus, Artifactory)

Build Lifecycle & Phases

- Default - compile, test, package, install, deploy
  - Project deployment

- Clean – clean
  - Clean previous build artifacts

- Site – site
  - Generate project documentation

Common Commands

- mvn compile # Compiles source code
- mvn test # Runs unit tests
- mvn package # Creates JAR/WAR file
- mvn install # Installs to local repository
- mvn clean # Deletes target directory

Why Use Maven?

- Automatic Dependency Management
  - Downloads required libraries automatically
  - Handles transitive dependencies (libraries your dependencies need)
- Standardized Project Structure
  - Consistent across all Maven projects
  - Easy for new developers to understand
- Large Ecosystem
  - Thousands of plugins for various tasks
  - Integrates with CI/CD tools (Jenkins, GitHub Actions)
- Declarative Configuration
  - Define WHAT you want, not HOW to do it
- Built-in Best Practices
  - Standardized testing, packaging, deployment

Quick Start Example

```
mvn archetype:generate -DgroupId=com.example \
                -DartifactId=my-app \
                -DarchetypeArtifactId=maven-archetype-quickstart \
                -DinteractiveMode=false
```

Hibernate ORM

- Hibernate ORM is a powerful open-source framework for Java that maps object-oriented domain models to relational databases.

- It serves as a bridge, allowing developers to interact with a database using standard Java objects rather than manual SQL queries.

- Hibernate addresses the **Object-Relational Impedance Mismatch**, which refers to the conceptual differences between how data is structured in Java (as objects and references) and how it is stored in databases (as tables and foreign keys).

Core Functionality

- Automatic Mapping: It automatically maps Java classes to database tables and Java data types to SQL data types.

- Standards Compliance: It is the primary implementation of the Jakarta Persistence (formerly JPA) specification, ensuring your code remains portable across different persistence providers.

- HQL (Hibernate Query Language): It provides an object-oriented version of SQL that works on persistent objects rather than tables, making queries database-independent.

Key Features (2026)

- Modern Support: As of early 2026, Hibernate 7.2 supports the latest Java runtimes (17, 21, and 25) and complies with Jakarta Persistence 3.2 and Jakarta Data 1.0.
- Performance Optimization: Includes built-in first-level and second-level caching to reduce redundant database hits.
- Database Independence: Applications can switch between different RDBMS (like MySQL, Oracle, or PostgreSQL) simply by changing a configuration "dialect".
- Lazy & Eager Loading: Controls when data is fetched—"Lazy" loading waits until data is actually accessed to save memory.

Architecture & Workflow

- Configuration: Reads settings from files like hibernate.cfg.xml or Java annotations.
- SessionFactory: A heavyweight, thread-safe object that initializes the framework and manages sessions.
- Session: A lightweight, non-thread-safe object representing a single unit of work (e.g., one database transaction).
- Transaction Management: Provides a consistent API to handle atomic operations (commit/rollback) across different environments.

Core Components & Their Roles

- Configuration Object: The first object created; it reads the hibernate.cfg.xml or properties file and mapping annotations to bootstrap the framework.
- SessionFactory (Heavyweight): A thread-safe singleton object that represents a single database. It acts as a factory for producing Session instances and holds an optional Second-Level Cache for improved performance.
- Session (Lightweight): A short-lived, single-threaded object representing a single "unit of work" or conversation with the database. It holds the First-Level Cache (mandatory) and is used for basic CRUD operations.
- Transaction Object: An optional component used to define atomic units of work (commit or rollback). It abstracts the application from the underlying JDBC or JTA transaction managers.
- Query & Criteria: These interfaces allow you to retrieve data using Hibernate Query Language (HQL), object-oriented criteria queries, or native SQL.

Typical Lifecycle of an Operation

- Load: The application loads the Configuration and builds a SessionFactory once at startup.
- Open: A Session is opened when a database interaction is required.
- Transact: A Transaction is started to ensure data integrity.
- Execute: Queries (HQL/SQL) are performed via the Session.
- Finish: The Transaction is committed, and the Session is closed to release resources

Hibernate object life cycle

- Transient State : An object is in the transient state when it has just been instantiated using the new operator. It is a plain old Java object (POJO), not associated with any Session or persistence context, and has no representation in the database.

- Persistent State : An object enters the persistent (or managed) state when it is associated with an active Hibernate Session and represents a row in the database.

- Detached: The object becomes detached when the Session is closed, cleared

- Removed State : An object is in the removed state when a persistent object is passed to the session.delete() or session.remove() method.

XML-Based Mapping (.hbm.xml)

```
<hibernate-mapping>
   <class name="com.example.Student" table="STUDENT">
      <id name="id" column="id">
         <generator class="increment"/>
      </id>
      <property name="name" column="student_name"/>
   </class>
</hibernate-mapping>
```

Annotation-Based Mapping (JPA)

```
@Entity
@Table(name="STUDENT")
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    @Column(name="student_name")
    private String name;
}
```

Performing CRUD Operations (Java Code)

- **CREATE (Insert)**

```
Transaction tx = session.beginTransaction();
Student s1 = new Student("Alice");
session.persist(s1); // or session.persist(s1)
tx.commit();
```

- **READ (Select)**

```
Student s = session.find(Student.class, 1);
System.out.println(s.getName());
```

- **UPDATE**

```
Transaction tx = session.beginTransaction();
Student s = session.find(Student.class, 1);
s.setName("Alice Updated");
session.update(s); // or session.merge(s)
tx.commit();
```

- **DELETE**

```
Transaction tx = session.beginTransaction();
Student s = session.find(Student.class, 1);
if (s != null) {
    session.delete(s); // or session.remove(s)
}
tx.commit();
```

The Problem of Granularity

- The Problem of Granularity is a key part of the "Object-Relational Impedance Mismatch." It occurs because the object-oriented world (Java) and the relational world (SQL) represent data at different levels of detail.

- Object Model (Fine-grained): Java encourages creating many small, specialized classes to improve code reuse and logic (e.g., a User class containing an Address object).

- Relational Model (Coarse-grained): Databases typically support only a few data types (VARCHAR, INT, etc.). There is no native "Address" type in standard SQL that matches a Java class; instead, data is stored in tables and columns.

Solving Granularity with Annotations

- Hibernate uses specific annotations to bridge this gap, allowing you to maintain a fine-grained object model while keeping a coarse-grained (and efficient) database schema.

- @Embeddable: You place this on the "fine-grained" class that doesn't have its own table (e.g., Address).

- @Embedded: You place this on the field in the main entity (e.g., User) where the object is used.

Java Code (Annotation Solution):

```
@Embeddable
public class Address {
   private String street;
   private String city;
}

@Entity
@Table(name = "USERS")
public class User {
   @Id
   private int id;
   @Embedded  // Solves the granularity problem
   private Address address;
}
```

Relationship

- Relationships define how Java objects (entities) link to one another, mirroring how tables are associated in a relational database.

Unidirectional vs. Bidirectional

- A relationship can be unidirectional (only one entity has a reference to the other) or bidirectional (both entities have references to each other).

- @OneToOne: A relationship where one instance of an entity is linked to exactly one instance of another entity (e.g., a Person and a Passport).

- @OneToMany: A relationship where one entity instance can be associated with multiple instances of another entity (e.g., a Department with multiple Employees).

- @ManyToOne: A relationship where multiple instances of an entity are linked to a single instance of another entity (e.g., many Employees belong to one Department).

- @ManyToMany: A relationship where multiple instances of one entity can be associated with multiple instances of another entity (e.g., many Students can enroll in many Courses).

@OneToOne

- The @OneToOne annotation in Hibernate/JPA establishes a one-to-one relationship between two entities.

- It means that one instance of an entity is associated with exactly one instance of another entity.

Unidirectional One-to-One

```
public class User {
  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;
   private String name;
  @OneToOne
   @JoinColumn(name = "address_id")
   private Address address;
      // getters and setters
```

```
@Entity
}
@Entity
public class Address {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;
   private String city;
   private String street;
}
```

Bidirectional One-to-One

```
@Entity
public class User {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;

   private String name;

   @OneToOne(mappedBy = "user")
  // Address owns the relationship
   private Address address;

   // getters and setters
}
```

```
@Entity
public class Address {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;

   private String city;

   @OneToOne
   @JoinColumn(name = "user_id")
  // Foreign key in Address table
   private User user;  // This side owns the relationship

   // getters and setters
}
```

@OneToMany

- The @OneToMany annotation in Hibernate/JPA establishes a one-to-many relationship between two entities, where one entity instance can be associated with multiple instances of another entity.

- Example: One Department has many Employees
- Example: One Customer has many Orders
- Example: One BlogPost has many Comments

Parent Entity (One side)

```
@Entity
public class Department {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @OneToMany(mappedBy = "department")
    private List<Employee> employees = new ArrayList<>();

    // getters, setters, constructors
}
```

Child Entity (Many side)

```
@Entity
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @ManyToOne
    @JoinColumn(name = "department_id")
    private Department department;

    // getters, setters, constructors
}
```

Key Attributes of @OneToMany

- fetch
  - Defines loading strategy

```
@OneToMany(mappedBy = "department", fetch = FetchType.LAZY)
private List<Employee> employees;
```

- cascade
  - Defines cascade operations from parent to children

- mappedBy
  - Specifies the field in the child entity that owns the relationship
  - Must be used on the non-owning side (parent side)

```
@OneToMany(mappedBy = "department", cascade = {
    CascadeType.PERSIST,
    CascadeType.MERGE,
    CascadeType.REMOVE
})
private List<Employee> employees;
```

```java
@Entity
public class Department {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;
   private String name;
    @OneToMany(mappedBy = "department",
            cascade = CascadeType.ALL,
            orphanRemoval = true)
   private List<Employee> employees = new ArrayList<>();

      public void addEmployee(Employee employee) {
      employees.add(employee);
      employee.setDepartment(this);
   }
      public void removeEmployee(Employee employee) {
      employees.remove(employee);
      employee.setDepartment(null);
   }
   }
```

```java
@Entity
public class Employee {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;

   private String name;

   @ManyToOne(fetch = FetchType.LAZY)
   @JoinColumn(name = "department_id")
   private Department department;

   // getters, setters
}
```

@ManyToOne in Hibernate

- The @ManyToOne annotation in Hibernate/JPA establishes a many-to-one relationship where multiple entities can reference a single entity.

- It represents the "many" side of a relationship.

  - Example: Many Employees belong to One Department

  - Example: Many Orders belong to One Customer

  - Example: Many Comments belong to One BlogPost

```java
@Entity
public class Book {
    @Id
    @GeneratedValue
    private Long id;

    private String title;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "author_id")
    private Author author;

    // Constructor
    public Book(String title, Author author) {
        this.title = title;
        this.author = author;
    }
}
```

```java
@Entity
public class Author {
    @Id
    @GeneratedValue
    private Long id;

    private String name;

    @OneToMany(mappedBy = "author")
    private List<Book> books = new ArrayList<>();
}
```

```
// Child entity
@Entity
public class Comment {
    @ManyToOne
    @JoinColumn(name = "post_id")
    private Post post;
}

// Parent entity
@Entity
public class Post {
    @OneToMany(mappedBy = "post")
    private List<Comment> comments = new ArrayList<>();

    // Helper method
    public void addComment(Comment comment) {
        comments.add(comment);
        comment.setPost(this);
    }
}
```

@ManyToMany

- A @ManyToMany relationship exists when multiple entities on both sides can be associated with multiple entities on the other side.

- Hibernate implements this using a join table (junction/association table) in the database.

```java
@Entity
public class Student {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;

   private String name;

   @ManyToMany
   @JoinTable(
      name = "student_course",
      joinColumns = @JoinColumn(name = "student_id"),
      inverseJoinColumns = @JoinColumn(name = "course_id")
   )
   private Set<Course> courses = new HashSet<>();

   // constructors, getters, setters
}
```

```java
@Entity
public class Course {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;

   private String title;

   @ManyToMany(mappedBy = "courses")
   private Set<Student> students = new HashSet<>();

   // constructors, getters, setters
}
```

Database Schema

students	student_course	courses
id (PK)	student_id (FK)	id (PK)
name	course_id (FK)	title

Key Attributes of @ManyToMany

```
@ManyToMany(
    cascade = {CascadeType.PERSIST, CascadeType.MERGE},
    fetch = FetchType.LAZY,  // Always use LAZY for performance!
    targetEntity = Course.class
)
```

Inheritance strategy

- Single Table Strategy (@Inheritance(strategy = InheritanceType.SINGLE_TABLE))
  - All classes in the hierarchy are mapped to a single database table, with a discriminator column (specified by @DiscriminatorColumn) to identify the specific subclass.

- Joined Strategy (@Inheritance(strategy = InheritanceType.JOINED))
  - Each class is mapped to its own table. Subclass tables store only their specific fields and share the primary key with the superclass table, which holds all common fields.

- Table Per Class Strategy (@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS))
  - Each concrete subclass is mapped to a separate, complete table that includes all inherited properties. There is no table for the abstract superclass.

Inheritance in Hibernate

- Hibernate provides several strategies to map inheritance hierarchies from object-oriented models to relational databases.

- Each strategy has its own trade-offs between normalization, performance, and complexity.

  - Single Table (SINGLE_TABLE) - All classes in hierarchy mapped to one table

  - Joined Table (JOINED) - Each class maps to its own table

  - Table Per Class (TABLE_PER_CLASS) - Each concrete class maps to its own table

```
@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "vehicle_type", discriminatorType = DiscriminatorType.STRING)
public class Vehicle {
  @Id
  @GeneratedValue private Long id;
   private String manufacturer;
}
@Entity
@DiscriminatorValue("BIKE")
public class TwoWheeler extends Vehicle {
   private String steeringHandleType;
}
@Entity
@DiscriminatorValue("CAR")
public class FourWheeler extends Vehicle {
   private String steeringWheelType;
}
```

- In this example, one table named Vehicle is created with columns: id, manufacturer, vehicle_type, steeringHandleType, and steeringWheelType.

```
@Entity
@Inheritance(strategy = InheritanceType.JOINED)
public class Account {
   @Id
   @GeneratedValue private Long id;
   private Double balance;
}
@Entity
public class CreditAccount extends Account {
   private Double creditLimit;
}
@Entity
public class DebitAccount extends Account {
   private Double overdraftFee;
}
```

- Hibernate will create three tables: Account (id, balance), CreditAccount (credit_id, creditLimit), and DebitAccount (id, overdraftFee).

```
@Entity
@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)
public abstract class Payment {
   @Id
   @GeneratedValue(strategy = GenerationType.SEQUENCE) // IDENTITY is not allowed
   private Long id;
   private Double amount;
}

@Entity
public class CreditCardPayment extends Payment {
   private String cardNumber;
}

@Entity
public class CashPayment extends Payment {
   private String shopLocation;
}
```

- Hibernate will create two tables: CreditCardPayment (id, amount, cardNumber) and CashPayment (id, amount, shopLocation).

HQL - Hibernate Query Language

- Hibernate Query Language (HQL) is an object-oriented query language that allows developers to write queries in terms of Java classes and properties rather than database tables and columns.

- While HQL looks similar to SQL, it operates on a higher level of abstraction, letting the Hibernate framework handle the translation into the specific SQL dialect required by your database.

Key Features of HQL

- Database Independence: Because queries are written against Java entities, they remain portable across different databases (e.g., switching from MySQL to PostgreSQL) without needing modification.

- Object-Oriented Nature: HQL understands advanced concepts like inheritance, polymorphism, and associations (e.g., @OneToMany).

- Automatic Result Mapping: HQL queries return results directly as Java objects or lists of objects, eliminating the need to manually extract data from a ResultSet.

- Case Sensitivity: HQL keywords (like SELECT, FROM, WHERE) are not case-sensitive, but the names of Java classes and their properties are case-sensitive.

Common HQL Clauses

- FROM: Loads complete persistent objects into memory. (e.g., FROM Student is equivalent to SELECT * FROM Student_Table).
- SELECT: Used when you only need specific properties rather than the entire object.
- WHERE: Filters results based on specific criteria.
- ORDER BY: Sorts the result set by one or more object properties.
- GROUP BY & HAVING: Groups data for aggregate operations like SUM, AVG, COUNT, MIN, and MAX.

```
Session session = sessionFactory.openSession();

// 2. Write the HQL (using class name "Employee", not table name)
String hql = "FROM Employee E WHERE E.salary > :minSalary ORDER BY E.lastName ASC";

// 3. Create the query and set parameters
Query<Employee> query = session.createQuery(hql, Employee.class);
query.setParameter("minSalary", 50000.0);

// 4. Execute and get results as a List of objects
List<Employee> results = query.getResultList();
```

Examples

```
SelectionQuery<Book> query = session.createSelectionQuery("SELECT b FROM Book b", Book.class);
List<Book> books = query.getResultList();

MutationQuery query = session.createMutationQuery(
   "UPDATE Product p SET p.price = p.price * 1.1 WHERE p.category = :cat"
);
query.setParameter("cat", "Electronics");
int updatedCount = query.executeUpdate();

MutationQuery query = session.createNativeMutationQuery(
   "UPDATE users SET status = 'INACTIVE' WHERE last_login < :cutoffDate"
);
query.setParameter("cutoffDate", someDate);
int rowsUpdated = query.executeUpdate();
```

```
@NamedQuery(
    name = "UpdateUserStatus",
    query = "UPDATE User u SET u.active = :status WHERE u.id = :userId"
)


MutationQuery mq = session.createNamedMutationQuery("UpdateUserStatus");
mq.setParameter("status", true);
mq.setParameter("userId", 123L);
int affectedRows = mq.executeUpdate();
```

Aggregate Functions

- HQL supports standard functions like SUM, AVG, COUNT, MIN, and MAX. An example is provided below.

```
String hql = "SELECT COUNT(e), AVG(e.salary) FROM Employee e";
Object[] result = (Object[]) session.createQuery(hql).getSingleResult();
```

Update and Delete

- Perform bulk operations directly on the entity model. An example update query is shown below.

```
int updatedEntities = session.createQuery(
    "UPDATE Employee e SET e.salary = e.salary * 1.1 WHERE e.performance =
'EXCELLENT'"
).executeUpdate();
```

Named Queries

- Named Queries are static queries defined in advance, usually at the top of an entity class using annotations.
- They are "pre-compiled" when the SessionFactory is created, providing a performance advantage as the SQL translation happens only once.
- Key Advantage: Centralizes query logic at the entity level, making the code cleaner and easier to maintain.
- Best For: Frequently used, fixed queries (e.g., fetching a user by their unique email).

```
@Entity
@NamedQuery(name = "findUserByEmail", query = "FROM User u WHERE u.email = :email")
public class User { ... }

// Execution
List<User> users = session.createNamedQuery("findUserByEmail", User.class)
                    .setParameter("email", "john@example.com")
                    .getResultList();
```

Lazy Loading

- Lazy loading in Hibernate is a performance optimization strategy that delays the retrieval of associated data or properties from the database until they are explicitly accessed by the application code.

- This approach improves initial load times and reduces memory consumption by avoiding the loading of unnecessary data.

- When an entity is retrieved, instead of fetching all of its related entities immediately (eager loading), Hibernate returns a proxy object (or a "lazy" collection wrapper) in place of the actual data.

- This proxy knows how to get the data but does not contain it yet. The data is only loaded in a fresh database call when a method (like a getter or a collection's size() or iterator()) is invoked on the proxy.

Configuration and Defaults

- In modern Hibernate (version 3 and later), lazy loading is generally the default behavior for collections.
- Collections (@OneToMany, @ManyToMany): These are lazily loaded by default (fetch = FetchType.LAZY).
- Single-valued associations (@OneToOne, @ManyToOne): These are eagerly loaded by default (fetch = FetchType.EAGER).
- You can explicitly control the fetching strategy using the fetch attribute in your JPA annotations: @OneToMany(mappedBy = "author", fetch = FetchType.LAZY)

Caching

- In Hibernate, caching is a performance optimization technique that stores recently used data in memory to reduce the number of expensive database round-trips. By serving data from memory, it significantly improves application throughput and scalability.

First-Level Cache (Session Cache)

- Scope: Associated with the specific Session or EntityManager object.
- Lifecycle: Automatically enabled by default and cannot be disabled. It exists only as long as the session is open and is cleared when the session closes.
- Function: Within a single transaction, if you request the same entity twice, Hibernate returns the version stored in the First-Level Cache rather than querying the database again.

```
Session session = sessionFactory.openSession();

// First query - hits database
Employee emp1 = session.get(Employee.class, 1L);  // SQL: SELECT ...

// Second query - returns from L1 cache (same session)
Employee emp2 = session.get(Employee.class, 1L);  // NO SQL

session.close(); // L1 cache destroyed
```

Second-Level Cache (SessionFactory Cache)

- Scope: Associated with the SessionFactory , making it shared across all sessions and users.
- Lifecycle: Optional and disabled by default. It persists as long as the application is running.
- Function: If data is not found in the First-Level Cache, Hibernate checks this shared cache before hitting the database.
- Configuration: Requires a third-party provider. Common implementations include:
- JCache (Standard): The recommended standard for modern Hibernate (version 6+).
- Ehcache, Infinispan, or Hazelcast: High-performance caching libraries that plug into Hibernate.

Enabling Second Level Cache

```
<!-- Hibernate Core -->

<!-- Ehcache (Popular L2 Cache Implementation) -->
<dependency>
   <groupId>org.hibernate</groupId>
   <artifactId>hibernate-ehcache</artifactId>
   <version>5.6.15.Final</version>
</dependency>

<!-- Alternative: Infinispan -->
<dependency>
   <groupId>org.infinispan</groupId>
   <artifactId>infinispan-hibernate-cache-v53</artifactId>
   <version>13.0.10.Final</version>
</dependency>
```

Configuration

```
<properties>
    <!-- Enable L2 Cache -->
    <property name="hibernate.cache.use_second_level_cache" value="true"/>
        <!-- Cache Provider -->
    <property name="hibernate.cache.region.factory_class"
          value="org.hibernate.cache.ehcache.EhCacheRegionFactory"/>
        <!-- Enable Query Cache -->
    <property name="hibernate.cache.use_query_cache" value="true"/>
        <!-- Show cache statistics -->
    <property name="hibernate.generate_statistics" value="true"/>
    <property name="hibernate.cache.use_structured_entries" value="true"/>
        <!-- JPA 2.0 Shared Cache Mode -->
    <property name="javax.persistence.sharedCache.mode" value="ENABLE_SELECTIVE"/>
  </properties>
```

Entity Configuration

```
// Option 1: Using JPA @Cacheable annotation
@Entity
@Cacheable
@Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
public class Product {
    @Id
    private Long id;
    private String name;
    private BigDecimal price;

    // Collections need separate cache configuration
    @OneToMany(mappedBy = "product")
    @Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
    private Set<OrderItem> orderItems = new HashSet<>();
}
```

Thank You